

COURSE PROJECT PRESENTATION

Self-Supervised Prompt Optimization

Reproducing & Extending SPO with Ensemble, Curriculum & Memory Improvements

● **Muhammad Umar Niazi**
MSDS25018

● **Sana Hussain**
MSDS25058

● **Faiq Imran**
MSDS25007

Problem Statement & Motivation

THE PROBLEM

- ◆ LLMs are powerful — but prompt quality drastically affects their reasoning
- ◆ Manual prompt engineering requires expertise & trial-and-error
- ◆ Existing auto-optimization methods rely on ground truth labels or human feedback
- ◆ Ground truth is costly & unavailable in real-world open-ended tasks

KEY INSIGHT

Insight 1

Prompt quality manifests directly in model outputs — no external reference needed

Insight 2

LLMs can judge which of two outputs is better (LLM-as-judge)

Insight 3

Pairwise output comparison gives strong optimization signal

What is SPO? — Self-Supervised Prompt Optimization

SPO is a cost-efficient framework that discovers effective prompts for both closed and open-ended tasks **without requiring external reference** — deriving evaluation and optimization signals purely from LLM output comparisons.



Optimize

Generates new candidate prompts by analyzing the current best prompt and its outputs



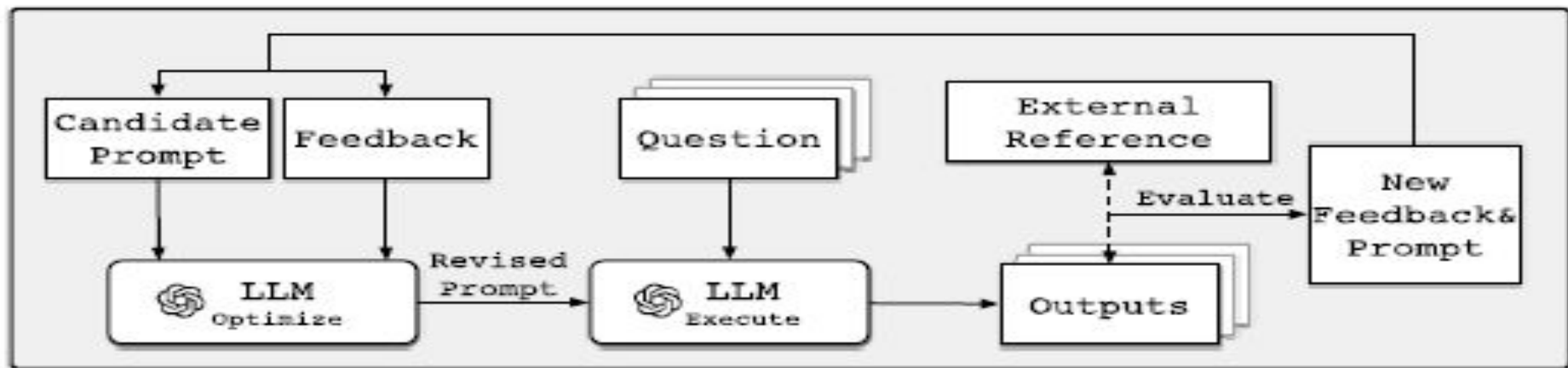
Execute

Applies prompts to sampled questions to produce model outputs

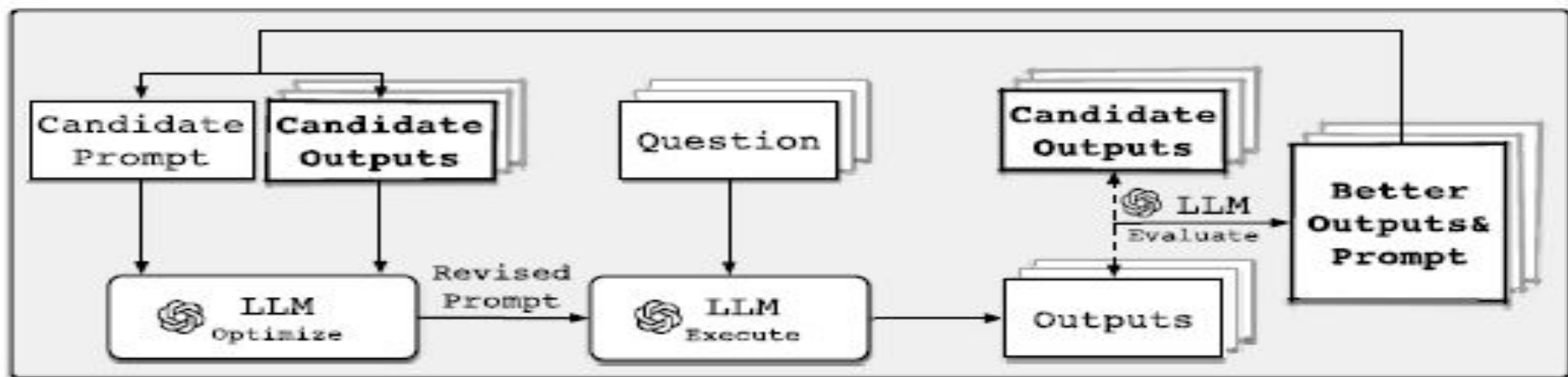


Evaluate

Pairwise LLM comparison selects the superior prompt without any ground truth



(a) Prompt Optimization With External Reference



(b) Self-Supervised Prompt Optimization (Ours)

Paper Figure 1: Prompt Optimization with vs. without External Reference

(a) With External Reference

Step 1: Candidate Prompt + Feedback → LLM Optimize

Step 2: LLM Optimize → Revised Prompt

Step 3: Question + Revised Prompt → LLM Execute

Step 4: LLM Execute → Outputs

Step 5: Outputs + **External Reference** → Evaluate

Step 6: Evaluate → New Feedback Prompt

Step 7: New Feedback Prompt → Feedback loop

⚠ *Requires labelled data or human evaluators at Step 5*

(b) Self-Supervised SPO (Ours)

Step 1: Candidate Prompt + Candidate Outputs → LLM Optimize

Step 2: LLM Optimize → Revised Prompt

Step 3: Question + Revised Prompt → LLM Execute

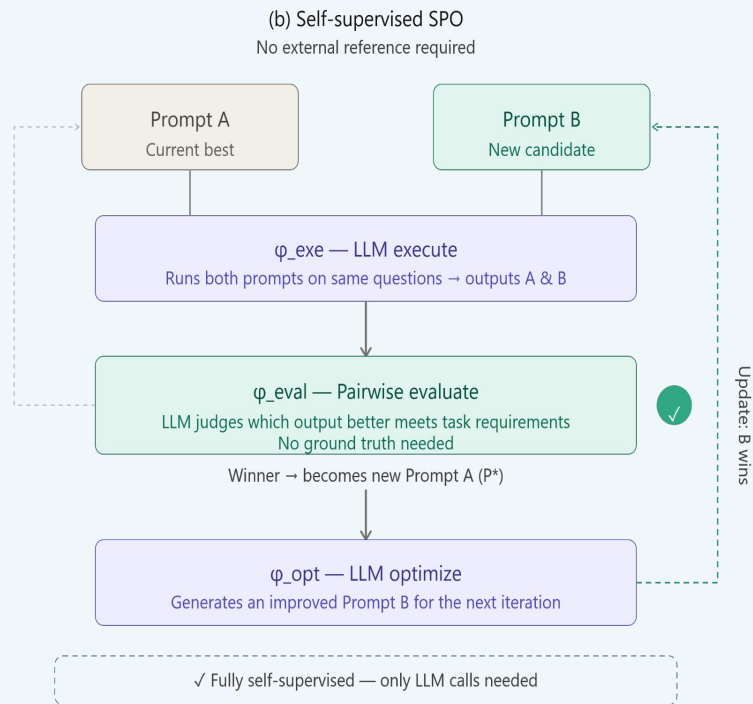
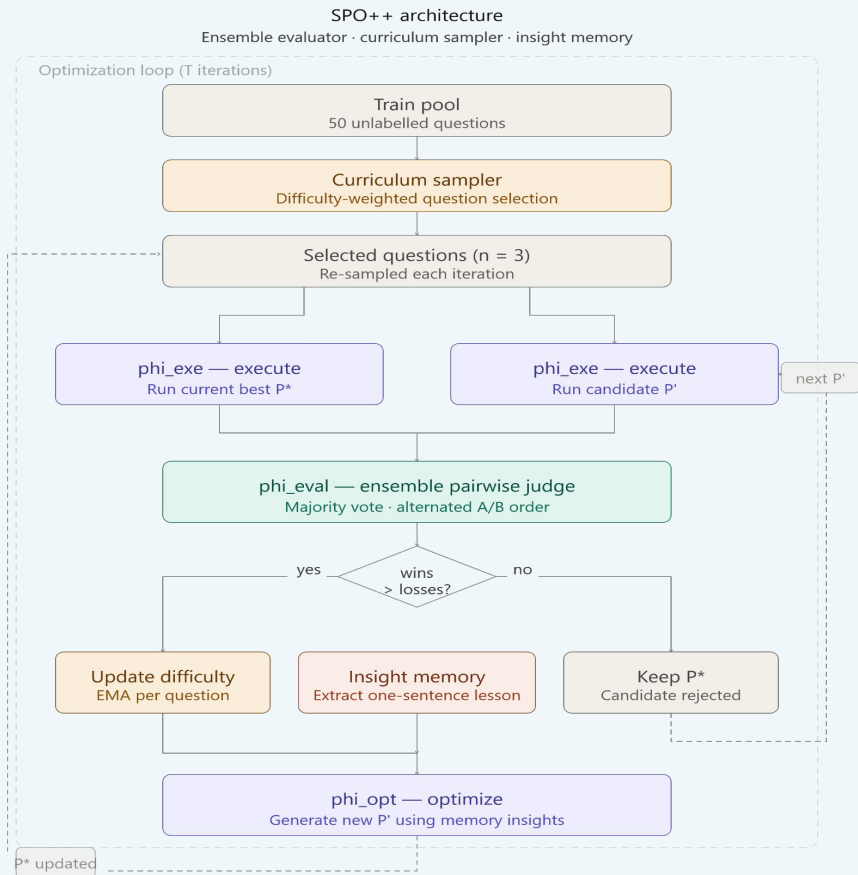
Step 4: LLM Execute → Outputs

Step 5: Candidate Outputs compared by **LLM Evaluate**

Step 6: Better Output selected → Better Outputs & Prompt

✓ *No external reference needed — LLM judges its own outputs*

Architecture



SPO Algorithm — Step by Step (Algorithm 1)

Algorithm 1: SPO

Require: Dataset D

Ensure: Optimized Prompt P^*

Init P_0 ; Sample 3 Qs from D

$P^* \leftarrow P_0$; $A^* \leftarrow \phi_{\text{exe}}(Q, P^*)$

for $\text{iter} \leftarrow 1$ to N_{max} do

$P' \leftarrow \phi_{\text{opt}}(P^*, A^*)$

$A' \leftarrow \phi_{\text{exe}}(Q, P')$

$\text{success} \leftarrow \phi_{\text{eval}}(Q, A', A^*)$

 if success then

$P^* \leftarrow P'$ $A^* \leftarrow A'$

 end if

end for

return P^*

1

Initialization

Start with a basic CoT prompt P_0 , sample 3 questions from the dataset, get initial outputs A^*

2

Optimize (ϕ_{opt})

LLM analyzes current best prompt P^* and outputs A^* , proposes an improved prompt P'

3

Execute (ϕ_{exe})

Run the new candidate prompt P' on the same questions to get new outputs A'

4

Evaluate (ϕ_{eval})

Pairwise comparison: if A' beats A^* , accept P' as the new best — otherwise keep P^*

SPO++ — Our Three Proposed Improvements

SPO Baseline (Algorithm 1)

1

Ensemble Evaluator

`ensemble_size = 3`

Instead of a single pairwise comparison call, 3 independent evaluator calls vote — majority wins. Reduces noise and positional bias in the evaluation step.

✓ More stable evaluation signal

2

Curriculum Sampling

`curriculum = True`

Questions are sampled proportionally to their average error-rate: harder questions get higher probability. Ensures the optimizer focuses on weaknesses, not just easy wins.

✓ Focuses on difficult cases

3

Meta-Prompt Memory

`memory = True`

After each iteration, a one-sentence insight is extracted (what changed, did it help?). The top-6 insights are injected into the optimization prompt for the next iteration.

✓ Avoids repeating past mistakes

Methodology & Experimental Setup

Task: BBH-Navigate (Big-Bench Hard — spatial navigation, Yes/No answers) | *Paper's running example & ablation benchmark*

Configuration

Optimization model	gpt-4o
Eval + Exec model	gpt-4o-mini
Iterations	10 (also tested: 20)
Samples / iter	3 (Algorithm 1 default)
Test set	60 questions (seed=42)
Train pool	50 questions
SPO++ ensemble	3 evaluator votes
Memory depth	Top-6 iteration insights

Model Pipeline



ϕ_{opt} (Optimizer)

GPT-4o temp=0.7



ϕ_{exe} (Executor)

GPT-4o-mini temp=0.0



ϕ_{eval} (Evaluator)

GPT-4o-mini temp=0.3

★ **Paper note:** Using GPT-4o as optimizer matches the paper's own SPO* ablation config (Table 1, avg 66.3%) — a validated alternative to Claude-3.5-Sonnet

Results — 10 Iterations (BBH-Navigate)

SPO (Baseline)



SPO++ (Our Extension)



Summary Table (10 iterations)

Method	Final Test Acc.	Successful Updates	gpt-4o calls	gpt-4o-mini calls
SPO (baseline)	98.3%	9 / 10	10	510
SPO++	96.7%	9 / 10	20	570

Results — 20 Iterations: Overfitting vs. Stability

SPO Baseline — 20 Iterations



SPO Baseline

Peaks at 100% (iter 6) but degrades to 95% by iter 20 — classic overfitting to the fixed 3-sample question set

SPO++ (Ensemble+Curriculum+Memory) — 20 Iterations



SPO++

Stays $\geq 95\%$ throughout; curriculum resampling prevents overfitting to specific questions and stabilizes performance

Paper Results — Performance vs. Cost Across Benchmarks

1.1%–5.6%

of competitors' cost

\$0.15

avg cost per dataset

Top-2

ranking across 5 benchmarks

3 samples

only, per iteration

Performance Comparison Across 5 Benchmarks (paper results)

Method	GPQA	AGIEval-MATH	LIAR	WSC	BBH-Nav	Avg
SPO ★	43.6	46.1	67.1	82.0	97.2	67.2
APE	40.8	41.0	63.8	76.8	91.3	62.7
OPRO	43.3	45.8	65.1	80.8	94.5	65.9
TextGrad	41.2	43.0	65.5	79.5	94.7	64.8
PromptBreeder	39.9	40.2	66.3	77.2	93.0	63.3

SPO consistently ranks top-2 across all benchmarks while costing only 1.1%–5.6% of what other methods spend. This makes SPO uniquely suited for real-world applications where labelled data is unavailable.

Analysis & Discussion



SPO baseline already near-perfect on BBH-Navigate

Starting accuracy of 98.3% leaves little room for improvement. The task may be too easy for this model configuration, explaining why SPO++ shows no gain here.



Curriculum sampling stabilizes long runs

At 20 iterations, SPO baseline degrades (overfits to same 3 questions). Curriculum's dynamic resampling prevents this — SPO++ stays consistently above 95%.



Ensemble adds cost without clear benefit here

Tripling evaluator calls (ensemble=3) increases gpt-4o-mini usage by ~12% but doesn't improve accuracy on a high-baseline task. May matter more on harder tasks.



Memory helps avoid repeated mistakes

The insight-extraction mechanism captures iteration-level lessons. Most useful when the optimizer is looping back to previously failed modifications.

Conclusion

01

Successfully reproduced SPO Algorithm 1 on BBH-Navigate, matching the paper's reported accuracy (98.3% vs paper's 97.2%)

02

Proposed SPO++ with Ensemble Evaluator, Curriculum Sampling, and Meta-Prompt Memory — three additive improvements to the baseline loop

03

Curriculum Sampling provides the clearest benefit: prevents overfitting at longer iteration counts and stabilizes performance

04

SPO's core insight holds: LLM outputs are rich enough to drive prompt optimization entirely without external supervision